APOSTILA DO ALUNO — AULA 13

Kickoff do Projeto: o App TechBurger

Curso: Técnico em Desenvolvimento de Sistemas | Turma: 103513 | Disciplina: Modelagem de Sistemas (82h + 04h planejamento)

Data: 27/10/2026 (terça-feira) | Horário: 07:30 às 11:30

Objetivos desta aula

- Entender como uma empresa de software organiza o trabalho: papéis, Scrum e o quadro Kanban.
- Escrever histórias de usuário com critério de aceite e conhecer a Definição de Pronto do time.
- Reconhecer atitudes de iniciativa e de autonomia no trabalho em equipe.
- Conhecer o projeto do app TechBurger: as 8 sprints, os 5 squads, as ferramentas e a avaliação.
- Entender as peças do Git e do GitHub: repositório, Issue, fork, branch, commit, pull request, review e merge.
- Fazer o primeiro pull request no repositório da empresa.

Como usar esta apostila

A partir desta aula a disciplina muda de formato: em vez de aulas expositivas, vamos trabalhar como uma empresa de software de verdade e construir, do zero, o app de pedidos do TechBurger. Esta apostila é o "manual do funcionário novo": explica como a empresa trabalha, o que vamos construir, como vocês serão avaliados e, na seção 9, o passo a passo do primeiro pull request. Guardem-na: ela vale para todas as sprints.

1. Bem-vindos à AprendendoDev

Durante 12 aulas vocês modelaram o TechBurger: regras de negócio, requisitos, casos de uso, classes, atividades, sequência, persona, wireframes, integrações e dados. Agora esse sistema vai virar um app de verdade, feito por vocês. Para isso, a turma passa a ser uma empresa: a AprendendoDev.

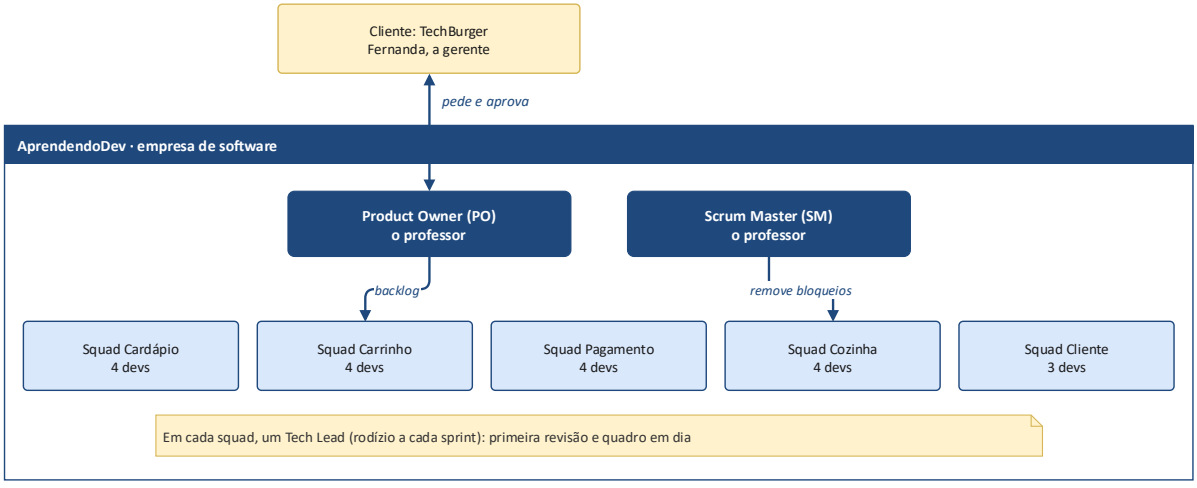


Figura 1 — A empresa AprendendoDev: o cliente, o PO, o Scrum Master e os 5 squads.

O CLIENTE é o TechBurger, representado pela gerente Fernanda. Ela conhece o negócio, não a tecnologia: fala de regras (“o pedido só vai para a cozinha com o pagamento confirmado”), não de código. O que ela pede vira requisito; o que ela aprova vira entrega. Nas sprints, o professor faz o papel dela nas entrevistas e nas demonstrações. Lembrem: o cliente não quer diagrama, quer o problema resolvido — o diagrama é uma ferramenta para NÓS não errarmos.

O que muda	Até a Aula 12	A partir da Aula 13
Formato da aula	Conteúdo, exemplo guiado e exercícios	Uma sprint por aula, com entrega no final
O seu papel	Aluno	Desenvolvedor (dev) de um squad
O papel do professor	Professor	Product Owner e Scrum Master
Onde fica o trabalho	Arquivos no seu computador	Repositório público no GitHub
O resultado	Diagramas de exemplo	O app TechBurger no ar, com o seu nome nas contribuições

O repositório é público: tudo o que vocês entregarem fica registrado, com o nome de cada um. É um portfólio real, que pode ser mostrado numa entrevista de estágio.

2. Como uma empresa de software trabalha

2.1 Por que um método de trabalho?

Sem método, todo mundo mexe em tudo, ninguém sabe o que falta e os erros só aparecem no final. No modelo CASCATA, tudo é planejado no começo e entregue de uma vez no fim; se o cliente muda de ideia no meio do caminho, muito trabalho se perde. Os métodos ÁGEIS fazem o contrário: entregam em pedaços pequenos, mostram ao cliente e ajustam a cada ciclo. O mais usado no mercado é o Scrum, quase sempre acompanhado de um quadro Kanban.

	Cascata	Ágil (Scrum)
Planejamento	Tudo no começo	Um ciclo de cada vez
Entrega	Uma vez, no fim	A cada sprint
Mudança de ideia do cliente	Custa caro	É esperada
Quando o cliente vê o sistema	Só no final	Em todas as sprints

2.2 Os 3 papéis do Scrum

Papel	O que faz	Na AprendendoDev
Product Owner (PO)	Decide O QUE será feito e em que ordem; é o dono do backlog e aceita as entregas	O professor
Scrum Master (SM)	Cuida do PROCESSO: conduz os eventos e remove os bloqueios do time	O professor
Time de desenvolvimento	Decide COMO fazer: modela, programa, revisa e entrega	Vocês, em 5 squads
Tech Lead do squad	Faz a primeira revisão dos PRs do squad e mantém o quadro em dia	Um dev de cada squad, em rodízio a cada sprint

2.3 O ciclo de uma sprint

Uma SPRINT é um ciclo curto de trabalho, sempre com o mesmo tamanho, que termina com algo pronto para mostrar. Na AprendendoDev, cada aula é uma sprint.

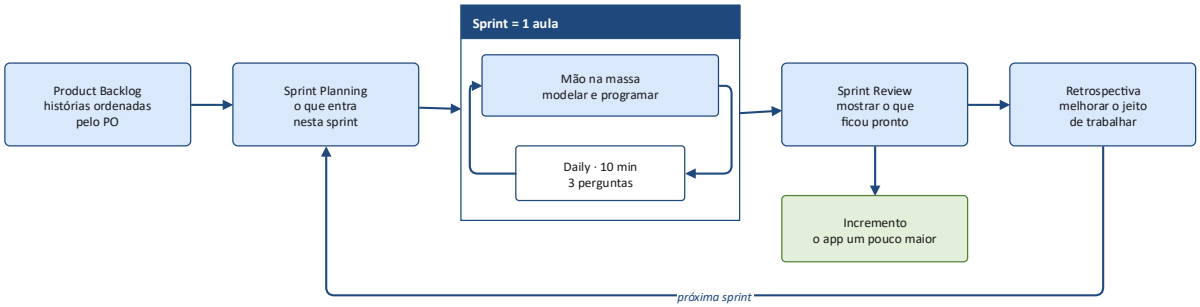


Figura 2 — O ciclo de uma sprint: backlog, planning, mão na massa com daily, review, retrospectiva e incremento.

Peça	O que é	Na nossa aula-sprint
Product Backlog	Lista ORDENADA de tudo o que o app precisa ter; cada item é uma história de usuário. O PO ordena por valor (o MoSCoW da Aula 2: Must primeiro).	Fica no quadro do GitHub, na coluna Backlog
Sprint Planning	Reunião em que o time escolhe o que entra na sprint e cada um pega a sua tarefa.	Início da aula, cerca de 20 minutos
Daily	Encontro de 10 minutos, em pé, com 3 perguntas: o que fiz? o que vou fazer? o que me bloqueia?	No meio da manhã, por squad
Sprint Review	Cada squad MOSTRA funcionando o que ficou pronto; o PO aceita ou devolve.	Fim da aula
Retrospectiva	O time olha para o PROCESSO (não para o código): o que funcionou, o que atrapalhou, o que vamos mudar.	Últimos minutos da aula
Incremento	Tudo o que foi aceito na review; soma-se ao que já existia.	O app cresce a cada aula; na Aula 21 vira a versão 1.0

Na daily, problema grande não se resolve na hora: quem tem um bloqueio fala dele em uma frase, e a conversa para resolver acontece logo depois, só com quem precisa participar.

3. História de usuário, critério de aceite e Definição de Pronto

3.1 História de usuário

A história de usuário é o requisito escrito do ponto de vista de quem usa — é o "requisito de usuário" da Aula 1, num formato curto, que cabe num cartão:

Como [quem], quero [o quê], para [por quê].

Exemplo: "Como cliente, quero ver o cardápio por categoria, para achar meu lanche mais rápido." O "para" é o mais importante: ele diz o VALOR da história e ajuda o PO a decidir a ordem do backlog. Uma boa história é pequena (cabe numa sprint e um dev consegue terminar), independente das outras e testável.

3.2 Toda história tem critério de aceite

O critério de aceite é o mesmo das Aulas 2 e 3: Dado / Quando / Então. Ele é o "contrato" entre o dev e o PO — na review, o PO confere exatamente o que está escrito ali.

História (Squad Carrinho)	Critério de aceite
Como cliente, quero ser avisado do pedido mínimo, para não perder tempo no pagamento.	Dado um carrinho com R$ 12,00, Quando tento finalizar o pedido, Então o app impede e mostra "Faltam R$ 3,00 para o pedido mínimo".

3.3 Definição de Pronto (Definition of Done)

A Definição de Pronto é a lista de tudo o que uma tarefa precisa cumprir para ser aceita. Ela vale para todos, em todas as sprints. Na AprendendoDev, uma tarefa só está pronta quando:

- faz o que o critério de aceite pede — e foi testada no navegador;
- segue o diagrama e os nomes do modelo (classes, telas, campos);
- tem pull request ligado à Issue e revisado por um colega;
- não quebra o que já funcionava nem mexe em arquivo de outro squad sem combinar;
- foi aceita pelo PO na Sprint Review.

4. O quadro Kanban

O quadro mostra, a qualquer momento e para todos, em que pé está cada tarefa. Cada cartão é uma Issue do GitHub (seção 8) e anda da esquerda para a direita.

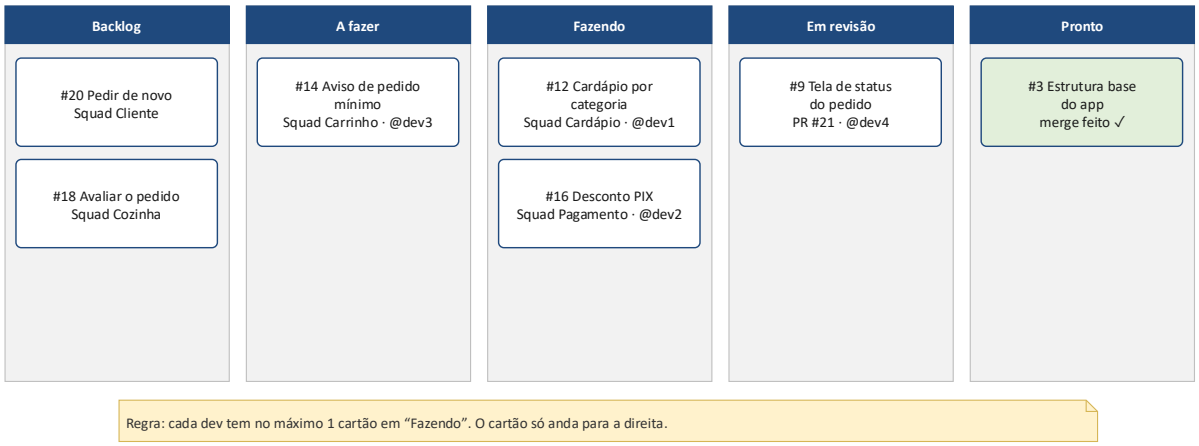


Figura 3 — O quadro Kanban do projeto.

Coluna	Significado
Backlog	Ainda não entrou em nenhuma sprint
A fazer	Escolhida para a sprint, mas ainda sem responsável
Fazendo	Alguém está trabalhando nela agora
Em revisão	Tem pull request aberto, esperando o colega e o professor
Pronto	Recebeu merge e o critério de aceite foi conferido

Regras do quadro: todo trabalho é um cartão (se não está no quadro, não existe); cada dev tem no máximo 1 cartão em “Fazendo”; e “Pronto” só depois do merge. Ninguém precisa arrastar cartão: o cartão anda sozinho. Quando o dev comenta /eu-pego na Issue, o robô do time entrega a tarefa a ele (Fazendo); quando o pull request é aberto, ela vai para Em revisão; com o merge, para Pronto. A página Sprint do portal mostra tudo isso em tempo real.

5. Iniciativa e autonomia no time

Num time ágil ninguém fica esperando ordem para cada passo. Duas atitudes fazem diferença — e serão observadas durante todo o projeto:

- **Iniciativa:** agir sem esperar mandarem — ver um problema e propor a solução, oferecer ajuda, abrir uma Issue para um bug que encontrou.
- **Autonomia:** decidir e resolver sozinho DENTRO do combinado, e pedir ajuda na hora certa.

Situação	Atitude esperada
Travei numa tarefa há 15 minutos	Pesquisar, perguntar ao squad e, depois, ao professor — e registrar a dúvida na Issue
Terminei antes do fim da sprint	Revisar o PR de um colega ou pegar o próximo cartão do backlog
Achei um erro no módulo de outro squad	Abrir uma Issue para aquele squad, e não corrigir por conta própria
Não entendi a história	Ler a Issue e o diagrama de referência; depois, perguntar ao PO
Vou faltar na próxima aula	Avisar o squad num comentário da Issue e combinar a entrega

Cuidado com o excesso: mudar o que é de outro squad sem combinar não é iniciativa, é atropelo — e costuma gerar conflitos no código.

6. O projeto: o app TechBurger

6.1 O que vamos construir

Um app web de pedidos: o cliente abre no celular ou no computador, vê o cardápio, monta o carrinho, paga (de forma simulada) e acompanha o pedido; a cozinha vê os pedidos pagos num painel. As 8 regras de negócio do TechBurger vão funcionar de verdade. No fim do projeto, o app fica no ar em aprendendodev.com.br/app.

Tecnologia	Para que serve no app
HTML	A estrutura de cada tela (títulos, botões, listas)
CSS	A aparência (cores, tamanhos, layout no celular)
JavaScript	O comportamento: somar o carrinho, aplicar as regras, mudar o status
JSON	Os dados: cardápio, pedidos e clientes (o mesmo formato da Aula 11)

Escolhemos HTML, CSS e JavaScript puros porque rodam em qualquer navegador, não exigem instalar nada, são a base de toda a web e podem ser publicados de graça no GitHub.

6.2 “Do zero”: primeiro modelar, depois construir

Não vamos copiar os diagramas das aulas anteriores. Cada squad vai levantar, modelar e construir o SEU módulo, usando as Aulas 1 a 12 como base de conhecimento. Isso é o que as empresas fazem: ninguém programa sem antes entender o problema.

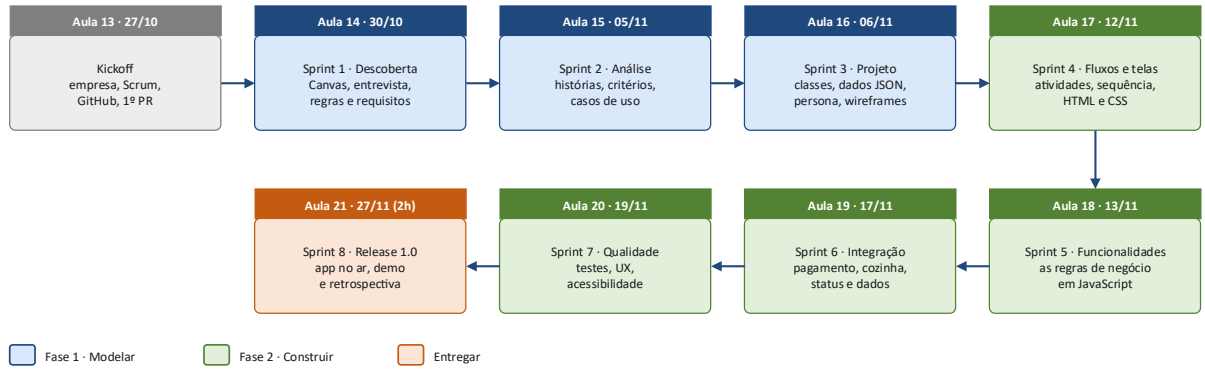


Figura 4 — O roadmap: kickoff + 8 sprints até a versão 1.0.

Aula	Sprint	Objetivo	Entregas principais
14 · 30/10	1 · Descoberta	Entender o negócio	Canvas do TechBurger, entrevista com a cliente, regras de negócio, RF e RNF
15 · 05/11	2 · Análise	Definir o que construir	Histórias com critério de aceite, casos de uso, backlog ordenado
16 · 06/11	3 · Projeto	Desenhar a solução	Classes e cardinalidade, dados em JSON, persona e wireframes
17 · 12/11	4 · Fluxos e telas	Do diagrama à tela	Atividades e sequência; telas em HTML e CSS
18 · 13/11	5 · Funcionalidades	As regras em código	Regras de negócio em JavaScript
19 · 17/11	6 · Integração	Os módulos conversando	Pagamento simulado, cozinha, status e dados
20 · 19/11	7 · Qualidade	Funcionar bem para todos	Testes dos critérios de aceite, UX, acessibilidade, correções
21 · 27/11 (2h)	8 · Release 1.0	Entregar	App no ar, demonstração de cada squad e retrospectiva final

6.3 Os 5 squads

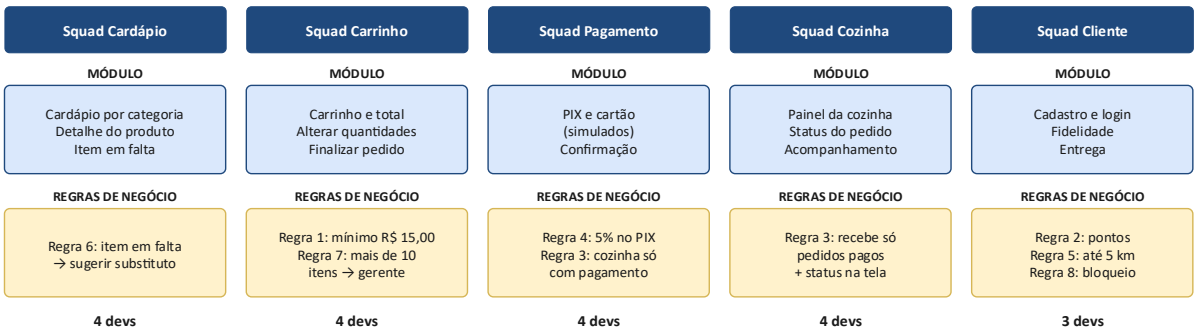


Figura 5 — Os 5 squads, os seus módulos e as regras de negócio de cada um.

Cada squad é dono de UM módulo do começo ao fim: quem modela o requisito é quem o programa. Cada squad tem os seus próprios arquivos (menos conflito, mais autonomia), e os módulos conversam entre si — o Carrinho usa os produtos do Cardápio, a Cozinha recebe os pedidos que o Pagamento confirmou. O professor anuncia hoje a composição de cada squad.

Regra de negócio	Squad responsável
1. Pedido pelo app só a partir de R$ 15,00	Carrinho
2. Fidelidade: 1 ponto a cada R$ 10,00; 100 pontos = combo grátis	Cliente
3. O pedido só vai para a cozinha com o pagamento confirmado	Pagamento (confirma) e Cozinha (recebe)
4. 5% de desconto no PIX pelo app	Pagamento
5. Entrega só até 5 km; fora disso, retirada no balcão	Cliente
6. Item em falta: sugerir um substituto	Cardápio
7. Mais de 10 itens: confirmação de um gerente	Carrinho
8. 3 cancelamentos no mês: bloqueio no app por 15 dias	Cliente

6.4 O rastro de uma regra: do diagrama ao código

Cada regra de negócio vai deixar um rastro por todo o projeto — é a rastreabilidade da Aula 9, agora até o código. Se a regra some em qualquer elo, o app erra.

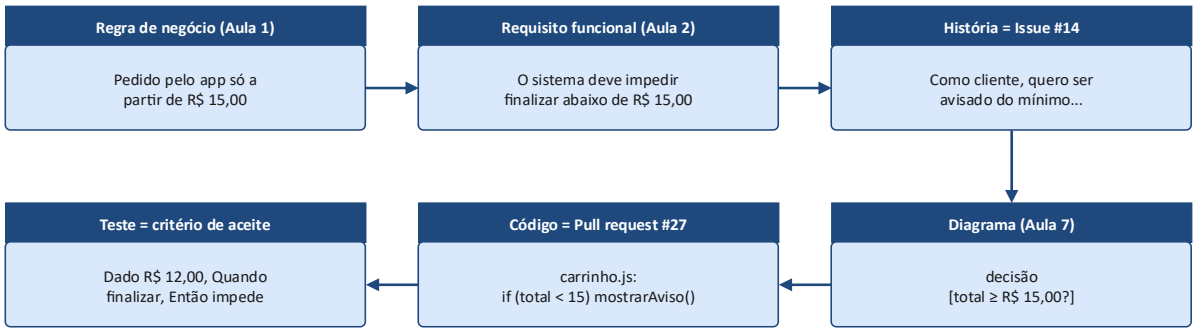


Figura 6 — O rastro da regra do pedido mínimo: regra, requisito, história, diagrama, código e teste.

7. Onde fica cada coisa, avaliação e regras do time

7.1 Portal, quadro e repositório

Lugar	Para quê
Portal · aprendendodev.com.br	Para se orientar: boas-vindas, regras do time, squads, objetivo da sprint, a página Começar (o seu card), “Minha tarefa” (o seu próximo passo) e guias passo a passo
Quadro · aba Projects do GitHub	As tarefas (Issues) e quem está com cada uma
Repositório · github.com/aprendendodev/techburger	Os diagramas (pasta docs/) e o código do app (pasta src/)
App publicado · aprendendodev.com.br/app	O resultado, atualizado a cada merge
Apostila de cada aula	O conteúdo da sprint e o passo a passo

Ficou com dúvida? Primeiro o squad, depois o professor — e registrem a dúvida (e a resposta) num comentário da Issue, para todos aproveitarem.

7.2 Como vocês serão avaliados

Critério	Peso	Como é medido
Entregas da sprint	40%	Issues concluídas e pull requests aceitos, sprint a sprint
Rastreabilidade	20%	Cada PR diz qual história resolve e qual diagrama segue
Participação no Scrum	20%	Planning, daily, review, retro e revisão do PR de um colega
Demonstração final do squad	20%	Aula 21: o módulo funcionando, apresentado pelo squad

O GitHub registra tudo: quem fez, quando fez e o que fez. Por isso a avaliação é individual, mesmo num trabalho em equipe.

7.3 E se eu faltar?

Cada falta é uma sprint perdida (e 4 faltas no diário). Mas o squad não fica travado: a sua tarefa volta para o backlog e pode ser assumida por outra pessoa. Você pode entregar de casa até o início da sprint seguinte — o GitHub funciona de qualquer lugar —, desde que avise o squad num comentário da Issue. Comunicação também é avaliada.

7.4 As regras do time

- Respeito: a crítica é ao código, nunca à pessoa.
- Cada um mexe no que é seu; para mexer no do outro, combine antes.
- Usou internet ou inteligência artificial? Tudo bem — mas saiba explicar cada linha que entregou.
- Nenhum dado pessoal no repositório: ele é público (nada de CPF, matrícula, telefone ou endereço).
- Pediu ajuda? Ajudou alguém? Registre na Issue — isso conta na participação.

8. Git e GitHub

Sem uma ferramenta própria, trabalhar em equipe no mesmo código vira bagunça: “app_final.html”, “app_final2.html”, arquivos trocados pelo WhatsApp, um apagando o trabalho do outro e ninguém sabendo o que mudou nem como voltar. A solução do mercado é o controle de versão.

	Git	GitHub
O que é	Programa de controle de versão	Site que hospeda repositórios Git
O que faz	Guarda o histórico de todas as alterações	Issues, pull requests, quadro, revisão e publicação de sites
Onde roda	No computador	Na internet (no navegador)

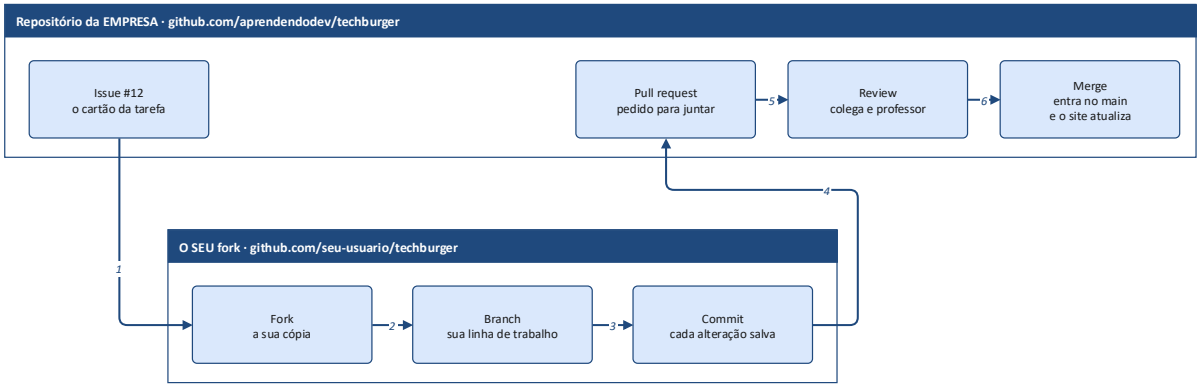


Figura 7 — O caminho de uma contribuição: do cartão (Issue) ao merge.

Peça	O que é	Exemplo no projeto
Repositório	A pasta do projeto, com todos os arquivos e TODO o histórico. A versão oficial fica no ramo principal, o main.	github.com/aprendendodev/techburger
Issue	O cartão de uma tarefa (ou de um bug), com número, responsável e etiquetas; é o cartão do quadro.	#12 Cardápio por categoria
Fork	A SUA cópia do repositório, na sua conta; feita uma vez só.	seu-usuario/techburger
Branch	Uma linha de trabalho separada para UMA tarefa; o main fica intacto.	carrinho-pedido-minimo
Commit	Uma alteração salva, com mensagem (verbo + o quê).	“Adiciona aviso de pedido mínimo”
Pull request (PR)	O pedido para levar as suas alterações ao repositório da empresa; diz qual Issue resolve.	Closes #12
Review	A revisão do PR: o Tech Lead e o professor aprovam, comentam ou pedem mudanças.	“Troque o nome da função para calcularTotal”
Merge	A junção do PR ao main; a Issue fecha e o site é atualizado.	Cartão vai para “Pronto”

Uma tarefa = uma branch = um pull request. Commits pequenos e com mensagem clara facilitam achar e desfazer um erro: “Adiciona aviso de pedido mínimo” diz tudo; “ajustes”, “aaa” e “agora vai” não dizem nada. Pedido de mudança no review não é bronca: é assim que as empresas garantem a qualidade — você ajusta na mesma branch e o PR se atualiza sozinho.

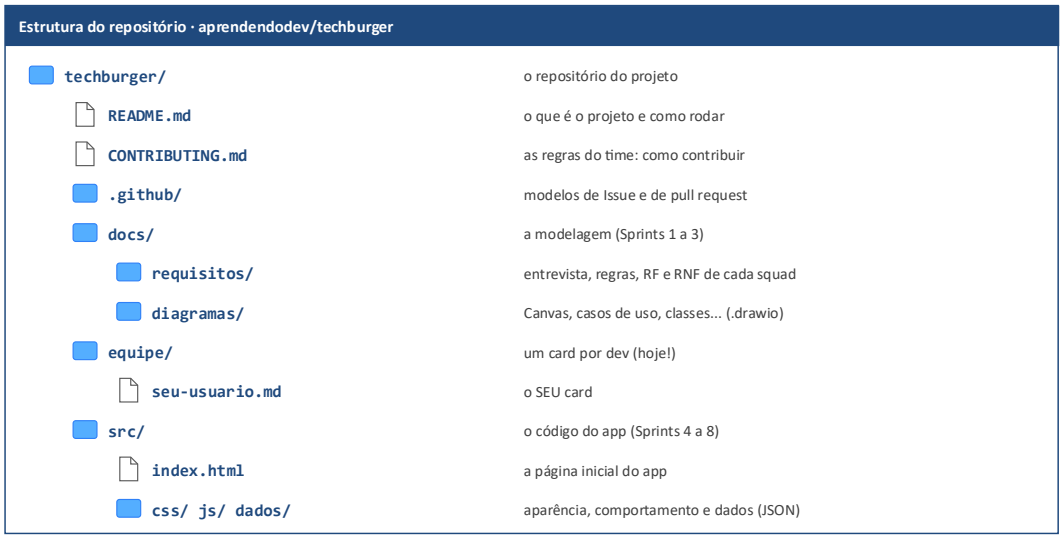


Figura 8 — A estrutura do repositório.

Ferramentas, sem instalar nada: o navegador com a sua conta do GitHub (Issues, quadro, PR e edição de arquivos); o github.dev, um editor de código dentro do navegador (aperte a tecla . na página do repositório); e o draw.io (app.diagrams.net), com os diagramas salvos como .drawio na pasta docs/. Em casa, quem quiser pode instalar o Git e o VS Code — o guia está no portal.

9. Passo a passo: o seu primeiro pull request

Cada dev vai criar o SEU card na pasta equipe/. Como cada um cria um arquivo próprio (seu-usuario.md), ninguém mexe no arquivo do outro — e não há conflito. Tudo pelo navegador: a página Começar do portal (aprendendodev.com.br/comecar.html) monta o card e traz dois botões: o 1 cria o seu fork (uma vez só) e o 2 abre o GitHub com o arquivo pronto; a branch o GitHub cria sozinho. As figuras são ESQUEMAS das telas do GitHub; os botões têm os mesmos nomes (em inglês), mas a aparência pode mudar um pouco.

O card tem este formato (a página Começar monta o texto a partir do formulário; o arquivo _modelo.md da pasta equipe/ traz o mesmo modelo, para quem preferir fazer à mão):

Linha do card	O que escrever
# Seu primeiro nome	Só o primeiro nome: o repositório é público
- GitHub: @seu-usuario	O seu usuário do GitHub
- Squad:	O seu squad (anunciado hoje)
- Quero aprender neste projeto:	Uma frase
- Uma curiosidade sobre mim:	Uma frase

Passo 0 — o ponto de partida

Entre no GitHub com a sua conta e abra github.com/aprendendodev/techburger. É o repositório da EMPRESA: você ainda não pode alterar nada aqui.

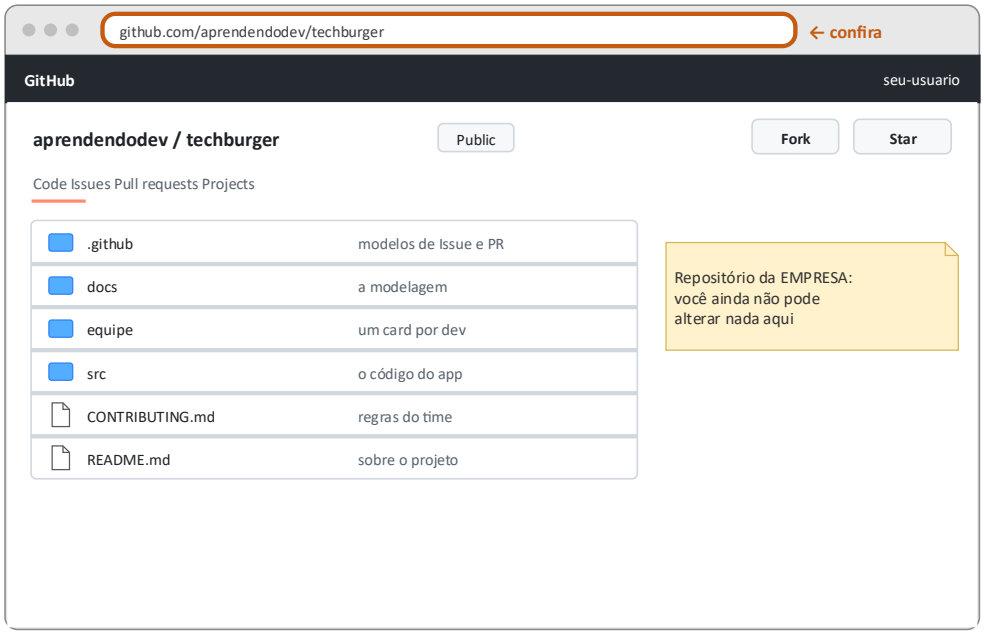


Figura 9 — Passo 0: o ponto de partida (esquema da tela).

Passo 1 — montar o card no portal

Abra aprendendodev.com.br e clique em Começar. Preencha o primeiro nome, o usuário do GitHub, o squad (o professor anuncia hoje; squad cheio fica bloqueado) e as duas frases. Confira a prévia: embaixo aparecem os botões 1. Criar o meu fork e 2. Criar o meu card no GitHub. Use primeiro o 1 e depois o 2.

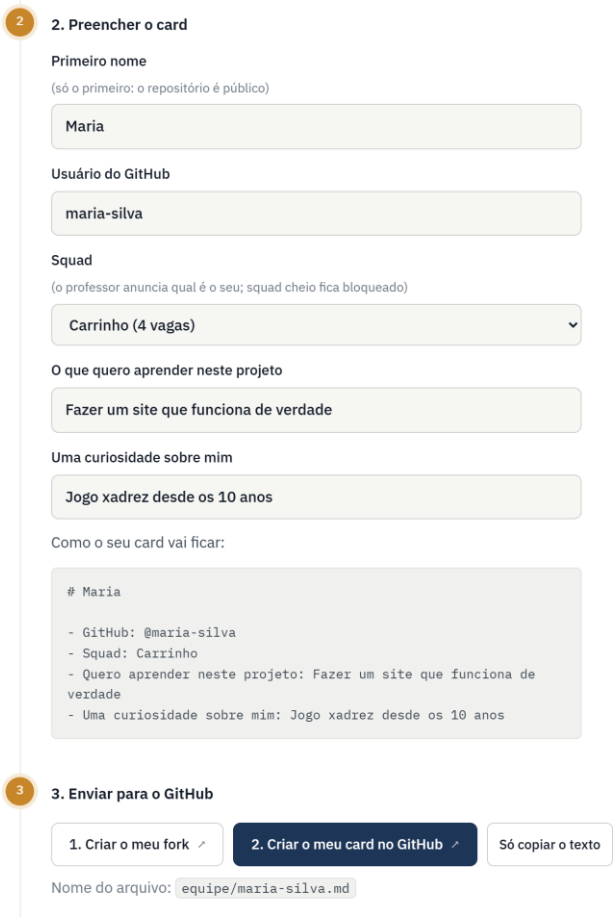


Figura 10 — A página Começar do portal, preenchida.

Passo 2 — fazer o fork

Clique em 1. Criar o meu fork: abre a página Create a new fork do GitHub. Deixe o dono (Owner) como o seu usuário e o nome techburger, e clique em Create fork. Agora existe seu-usuario/techburger, a SUA cópia. É uma vez só: o mesmo fork serve para todas as tarefas do projeto. Se nada acontecer ao clicar, o seu e-mail ainda não foi confirmado: abra o e-mail do GitHub (ou Settings › Emails), confirme e tente de novo.

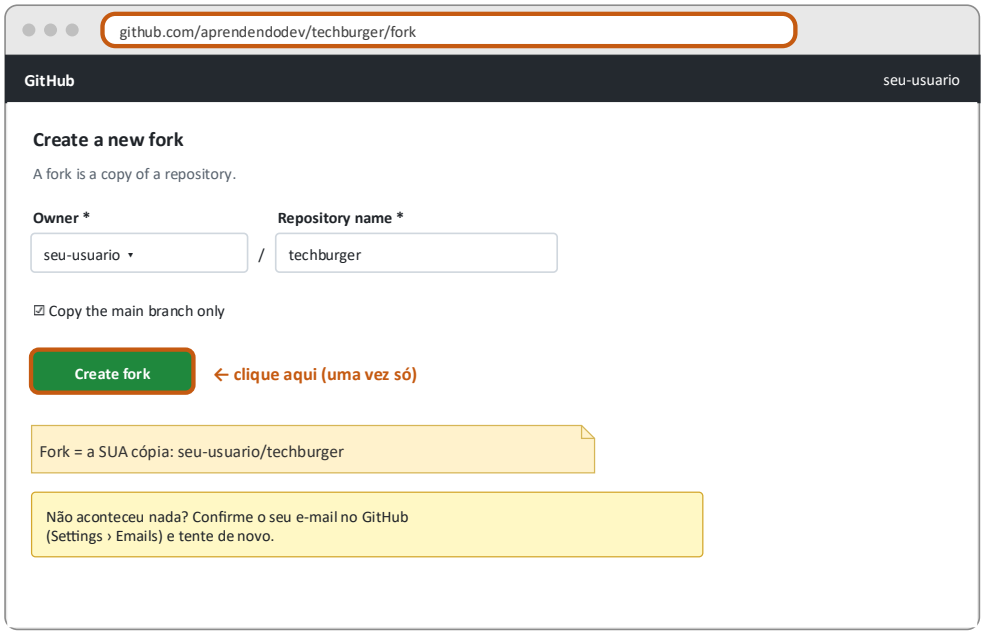


Figura 11 — Passo 2: fazer o fork (esquema da tela).

Passo 3 — conferir o seu arquivo

Volte ao portal e clique em 2. Criar o meu card no GitHub. O nome (seu-usuario.md) e o texto do card já vêm preenchidos (o texto também fica copiado). Confira o caminho techburger / equipe / seu-usuario.md e o squad. Se a caixa vier vazia, digite o nome do arquivo e cole o texto (Ctrl+V). Só o primeiro nome, nada de dado pessoal.

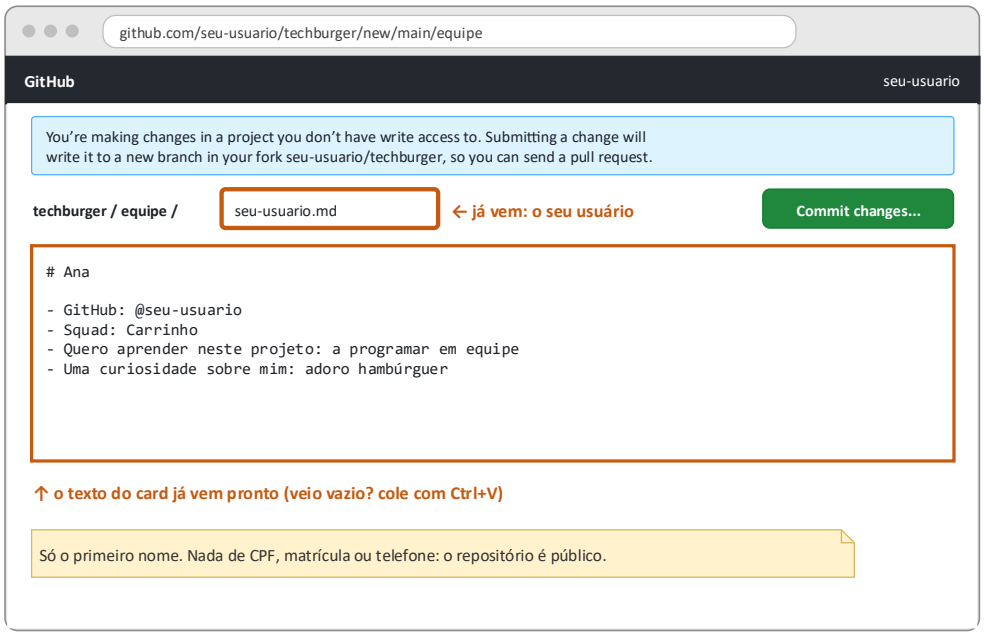


Figura 12 — Passo 3: conferir o seu arquivo (esquema da tela).

Passo 4 — salvar (commit)

Clique em Commit changes..., escreva a mensagem "Adiciona card de @seu-usuario" e clique em Propose changes. O GitHub cria uma branch no seu fork (patch-1).

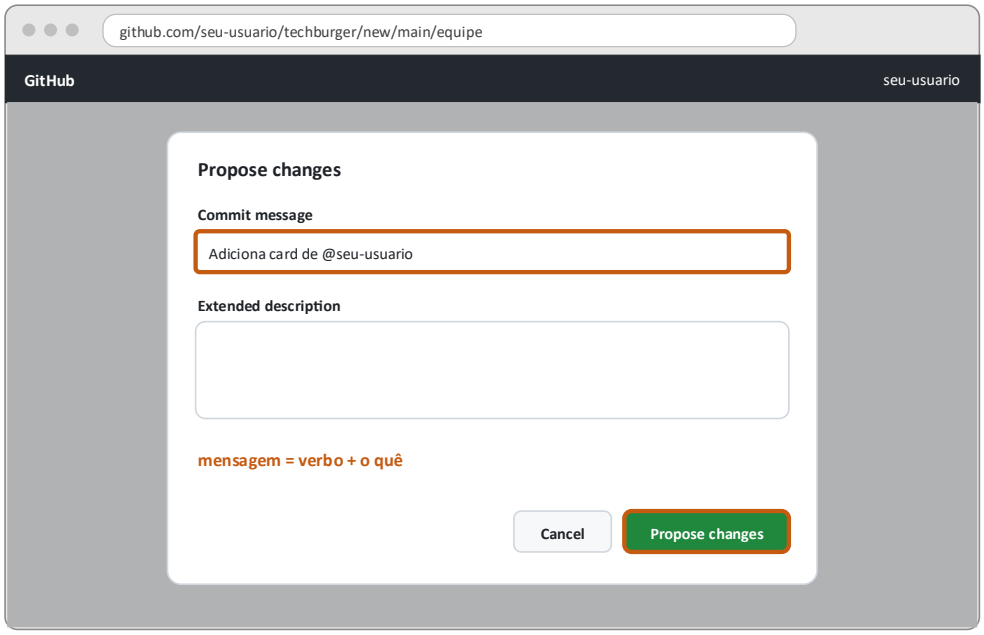


Figura 13 — Passo 4: salvar (commit) (esquema da tela).

Passo 5 — comparar as mudanças

Na página Comparing changes, confira: PARA aprendendodev/techburger (main), DE seu-usuario/techburger (patch-1), 1 arquivo novo, só o seu. Clique em Create pull request.

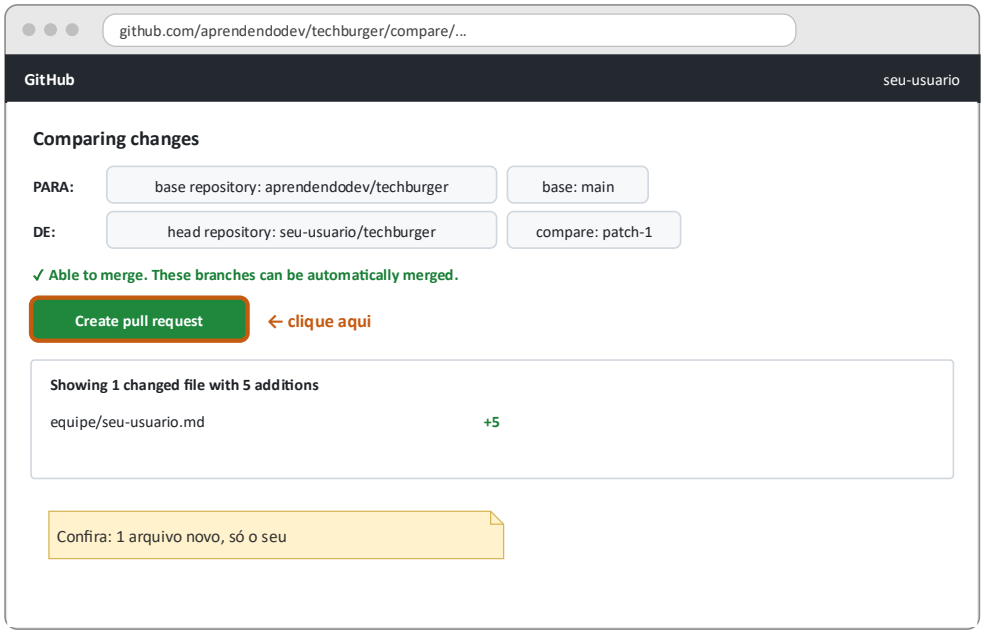


Figura 14 — Passo 5: comparar as mudanças (esquema da tela).

Passo 6 — descrever o pull request

Título: “Adiciona card de @seu-usuario”. A descrição já vem com o modelo do time: preencha. Na Issue, escreva “Parte de #1” — a #1 (Sprint 0) é de todos. Clique em Create pull request.

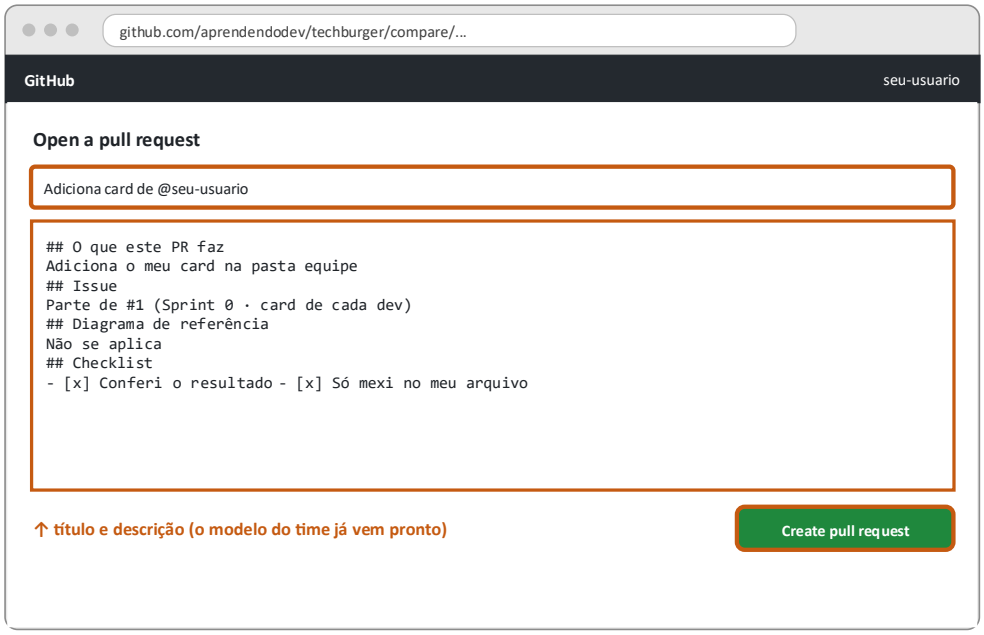


Figura 15 — Passo 6: descrever o pull request (esquema da tela).

Passo 7 — aguardar a revisão

O PR aparece como Open na aba Pull requests da empresa. O robô do time confere o card (squad, usuário) e comenta no PR; se você esqueceu o “Parte de #1”, ele coloca. O professor revisa: aprova ou pede mudança. Se pedir, edite o arquivo no SEU fork, na mesma branch (patch-1): o PR se atualiza sozinho. Depois do merge, o card aparece na página “Time” do portal.

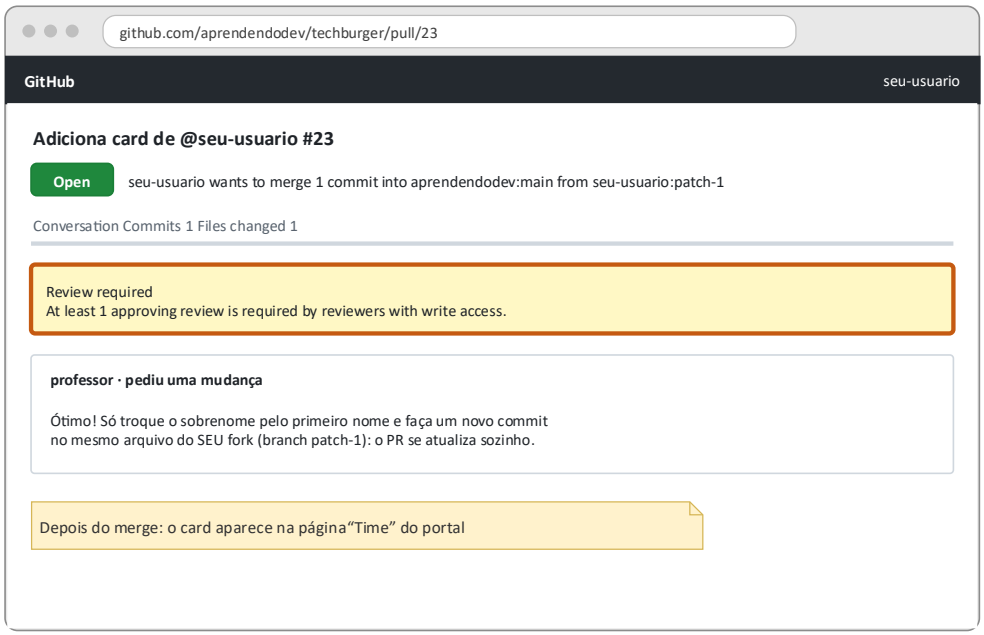


Figura 16 — Passo 7: aguardar a revisão (esquema da tela).

Por que “Parte de #1” e não “Closes #1”? Se alguém escrevesse “Closes #1”, o primeiro merge fecharia a Issue de todo mundo. Nas sprints, cada Issue será de UM dev, e aí o PR usa “Closes #número”: o merge fecha a Issue e o cartão vai sozinho para “Pronto”.

Não terminou na aula? Termine em casa até 30/10, seguindo estes passos. Se travar, comente na Issue #1 dizendo em qual passo parou.

10. Exercício: encontre os problemas

Em duplas, analisem a Issue e o pull request que o estagiário abriu para o Squad Carrinho. Há 6 problemas: 3 na Issue e 3 no PR. Para cada um, digam o que está errado e qual regra do time (seções 3, 4, 7 e 8) ele quebra, e escrevam a correção. Tempo: 8 minutos.



Figura 17 — A Issue #15 e o pull request #31 do estagiário.

11. Erros comuns de quem está começando

- Trabalhar sem Issue: “fiz uma coisa aí” não entra no quadro nem na avaliação.
- Tarefa grande demais: não termina na sprint — quebre em histórias menores.
- Mensagem de commit vaga e PR sem descrição.
- Mexer em arquivo de outro squad sem combinar: conflito na certa.
- Esperar o professor para tudo: primeiro tente, pesquise e pergunte ao squad.
- Colocar dado pessoal no repositório público.

12. Antes da Sprint 1 (30/10): checklist

- Conta no GitHub criada e card enviado pela página Começar.
- Card na pasta equipe/ com pull request aberto (ou já aceito).
- Saber o seu squad e o módulo dele (seção 6.3).
- Ler o README e o CONTRIBUTING do repositório.
- Rever as Aulas 1 e 2 (regras de negócio e técnicas de levantamento): a Sprint 1 começa com a entrevista com a cliente.

Glossário

Termo	Significado
Sprint	Ciclo curto de trabalho, com entrega no fim (na AprendendoDev: uma aula).

Product Owner (PO)	Decide o que será feito e em que ordem; dono do backlog.
Scrum Master (SM)	Cuida do processo e remove os bloqueios do time.
Squad	Pequeno time de devs, dono de um módulo do app.
Product Backlog	Lista ordenada de tudo o que o produto precisa ter.
História de usuário	Como [quem], quero [o quê], para [por quê].
Definição de Pronto	O que uma tarefa precisa cumprir para ser aceita.
Incremento	O que ficou pronto e foi aceito na sprint.
Kanban	Quadro de colunas que mostra o andamento de cada tarefa.
Repositório	A pasta do projeto, com todo o histórico de alterações.
Issue	O cartão de uma tarefa (ou de um bug) no GitHub.
Fork	A sua cópia do repositório, na sua conta.
Branch	Linha de trabalho separada para uma tarefa.
Commit	Alteração salva, com mensagem.
Pull request (PR)	Pedido para juntar as suas alterações ao repositório oficial.
Review / merge	A revisão do PR / a junção dele ao main.
Conflito	Duas mudanças na mesma linha do mesmo arquivo: o Git não sabe qual vale.

Perguntas de revisão

1. Qual a diferença entre o Product Owner e o Scrum Master?
2. Cite os 4 eventos de uma sprint e o objetivo de cada um.
3. Escreva uma história de usuário com critério de aceite para a regra da entrega até 5 km.
4. O que é a Definição de Pronto? Cite 3 itens da nossa.
5. Qual a diferença entre fork e branch?
6. Por que cada dev criou um arquivo próprio na pasta equipe/, e não uma linha num arquivo único?

Quiz relâmpago para casa

Respostas no Gabarito.

1. Em que coluna do quadro fica uma tarefa com pull request aberto, esperando revisão?
2. “Como cliente, quero pagar com PIX, para ganhar 5% de desconto.” Qual parte é o “para quê”?
3. Qual é a mensagem de commit melhor: “ajustes” ou “Adiciona aviso de pedido mínimo”? Por quê?
4. O que acontece com a Issue #12 quando o PR com “Closes #12” recebe o merge?
5. Faltei na Sprint 3. O squad fica travado esperando minha tarefa?